

AIに任せても、破綻しない。

再現性のある開発手法「DD」と、その実例 SynchroniNote

ナナイロ / 完全オフラインAI議事録 / 2026-06

- 1. 課題 — AI内製の落とし穴（バンプコーディング）
- 2. 開発手法 — DD + 複数LLMの適材適所運用
- 3. 実例：SynchroniNote — 完全オフラインAI議事録
- 4. ローカルLLMの今 — 性能・コスト・セキュリティ
- 5. まとめ — 経営へのメッセージ

このスライドで伝えたいこと（3行）

- ナナイロは、再現性のある自社手法 **DD** でAIを規律づけて開発している
- だから “バイブコーディング” に陥らず、**専門外の領域でも破綻しない** — 実例が完全オフラインAI議事録 **SynchroniNote**
- 土台にあるのは、**複数のLLMを適材適所で使いこなす**運用力

AI内製の落とし穴 = “バيبコーディング”

課題

開発手法

実例

ローカルLLM

まとめ

一言でいうと：AIに雰囲気を書かせて、動くけれど中身が誰にも分からないコードの山になること

⚡ 速い。デモは作れる

✗ でも：中身が黒箱／属人化／後で崩れる／品質は運任せ

💣 結果＝説明できない技術的負債。これは経営にとってのリスク

ここを越えられるかが、AI内製が“使い物になるか”の分かれ目

土台：複数のLLMを“適材適所”で使いこなす

課題

開発手法

事例

ローカルLLM

まとめ

ナナイロは「1つの万能AI」に頼らない。それぞれの個性を理解して使い分ける

💡 構想・壁打ち：**Gemini**（広い文脈・長文の設計ドラフト）／**ChatGPT**（発想・論点整理・別視点の検証）

✂ 実装・検証：**Claude (Claude Code)** = コードベースに直結し、実装・実測・UI確認まで回す主戦力

📌 上流は“発想力”のLLM、下流は“実装力”のLLM — この使い分け自体がナナイロの強み



3つのLLM（ChatGPT・Gemini・Claude）を目的別に使い分ける

「どのAIに、何をやらせるか」を設計できることが、AI活用力の本体

ナナイロの答え = DD（設計ドキュメント駆動）

課題

開発手法

実例

ローカルLLM

まとめ

DD は、AIを“半自動の実装者”として規律づける、ナナイロ自社製の開発手法

- 着手の前：「何を・なぜ・どう作り・何をもって完了とするか」を1枚にまとめる
- 実装の後：「実測値・決定理由・完了確認」を同じ紙へ書き残す
- 徹底：**1作業 = 1設計書 = 1チケット**

Claude Code 上の自作スキル /dd で、この一連を回している

どう回すか：構想から実装まで一本の流れ

課題

開発手法

事例

ローカルLLM

まとめ

- ① 壁打ち（Gemini / ChatGPT）で構想を立てる
- ② 企画書・開発ロードマップ = **基本設計** に落とす
- ③ ロードマップの各単位を **DDとして起票**（1作業 = 1枚）
- ④ **Claudeが実装 + 実測** — 動かして数字を取る
- ⑤ 検証してアーカイブ。判断の根拠が全部残る

証跡は 40件超のDD（DD-001～016）。すべて開発ロードマップ／DD索引から辿れる

なぜ破綻しないか = “実測が設計を上書きする”

課題

開発手法

実例

ローカルLLM

まとめ

最初の構想（壁打ちの初期案）の甘いところを、DDの実測ゲートが一個ずつ捕まえて直している

- 初期案「32GBをAIに／Qwen 32B」 → 実測：使えるRAMは約9.6GB → **gemma4:26b へ補正**
- 初期案「1～3分ごとにリアルタイム要約」 → 実測：CPUでSTTとLLMは同時に回らない → **終了後バッチへ撤回**
- 初期案「STT遅延1秒未満」 → 実測：RTF 0.527 → **一区切り約4秒に現実化**

「AIの言いなり」ではない。基本設計書は“要件と食い違えば実測を優先”と明文化 — これがバイブコーディングとの決定的な差

ドメイン知識は要らない。でも、エンジニアリング力是要る。

課題

開発手法

実例

ローカルLLM

まとめ

🟢 **床（下限保証）** : STT・オフラインLLMが“初耳”のエンジニアでも、手法に沿えば破綻しない（甘い前提は実測ゲートで落ちる）

🔧 **でも最後の仕上げ** — 判断・詰め・品質の作り込みは、**エンジニアリング力がある人にしか出来ない**

🚀 **天井（上限突破）** : エンジニアリング力が高いほど、精度は **他社を寄せ付けない**

DDは“エンジニアリング力の増幅器”。AIはエンジニアを置き換えず、できる人をさらに伸ばす

実例：SynchroniNote ① 会議の「前」

課題

開発手法

実例

ローカルLLM

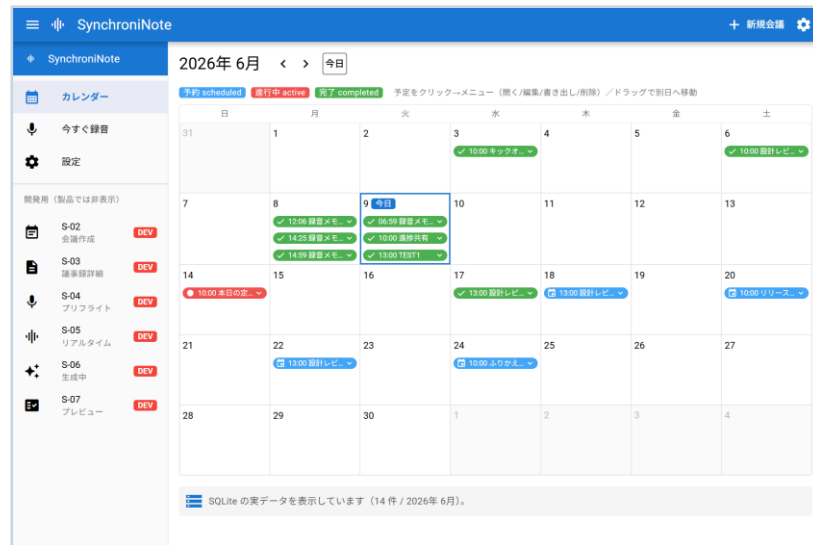
まとめ

専門外から着手して、破綻なく完成した“専門アプリ” = 手法の成果物（Exhibit A）

① カレンダーで予約・管理（予定の作成・移動・削除）

📄 予定をコピーで取込 — 案内メールを貼るだけでAIが会議情報に整形

🗂️ Excel / PDF を添付して要点を抽出、要約の文脈に



S-01 カレンダー



S-02 会議作成・コピー取込

実例 : SynchroniNote ② 会議の「中」 ★中核

課題

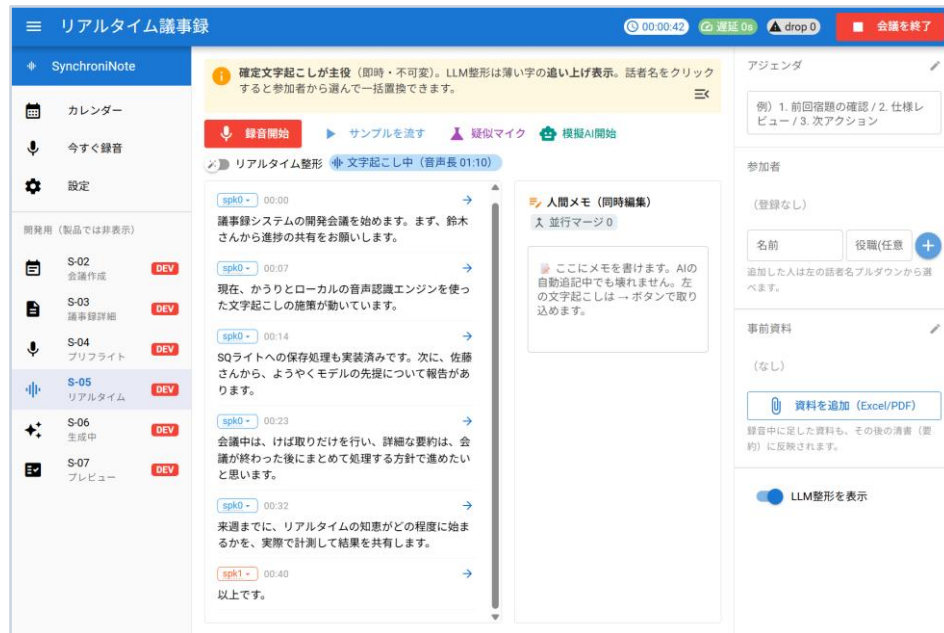
開発手法

実例

ローカルLLM

まとめ

- 🗣️ 話した先から文字起こし — 確定テキストは即・不変で表示（遅延ほぼ0）
- 👥 話者を自動で色分け — クリックで参加者名に一括置換
- 📝 人間メモを割り込み — その場の決定事項をタイムラインに差し込み
- 🔴 録音前にマイク・無音をチェックして“無音録音”の事故を防止



S-05 リアルタイム議事録（中核）



S-04 プリフライト

実例：SynchroniNote ③ 会議の「後」 + 破綻させない作り込み

課題

開発手法

実例

ローカルLLM

まとめ

🗨️ 終了ボタンで自動清書（決定事項・TODO・要約、目安3～6分）→ 保存でカレンダーに完了

🗨️ 詰まっても文字起こしは止まらない — 重い処理は遅らせ、主役は崩さない設計

👤 人間の確定 > AIの推測 — 話者名はクリック一発で総入れ替え



S-03 議事録詳細（最終議事録）

“動く”だけでなく“実際の会議で破綻しない”ところまで —
ここがエンジニアリングの仕事

AIは全部、この1台のPCの中で（完全ローカル）

課題

開発手法

実例

ローカルLLM

まとめ

- 文字起こし : whisper medium + VAD（無音を除くと日本語精度が約5倍改善）
- ライブ整形 : Ollama qwen3:8b / 終了後の清書 : Ollama gemma4:26b（MoE = 26B級の品質を4B級の速度で）
- 話者識別 : CAM++ ONNX 埋め込み（モデル未配置でも完全オフラインで代替動作）



S-08 設定（モデル・コア配分の実機チューニング）

クラウド送信ゼロ。すべてこのPCの中で完結する

ローカルLLMで今、何が起きているか

課題

開発手法

実例

ローカルLLM

まとめ

わずか1年で「小型モデルが、昨年の大型モデルに追いつく」時代に

- Gemma 4 (Google・2026年4月) — 「31Bが400B級に匹敵」と評価。本アプリ採用の26B MoE は活性わずか約3.8B = 大型の品質を小型の速度で
- Qwen3 / Qwen3.5 (Alibaba・Apache 2.0) — オープンモデルで Gemini 2.5 Pro と肩を並べる水準に
- 📈 オープンモデルとフロンティア最上位の差は わずか数% / ELO約53pt まで接近

「無料で持ち帰れる高性能モデル」が、毎月のように更新されている

正直な話：コストだけなら、クラウドが安い

課題

開発手法

実例

ローカルLLM

まとめ

☁ Gemini 2.5 Flash：入力 約\$0.30 / 出力 約\$2.50（100万トークンあたり）。会議1本でも数円～数十円、ハードも不要

💻 ローカルはマシン性能が必要（本アプリは既存PCで動かすので追加ハード費はほぼ0。ただし速度・メモリの制約は受ける）

⚖️ だから“コスト効率だけ”を見れば、クラウドAPIが有利な場面は多い — ここは隠さない

では、なぜそれでもローカルなのか？ → 次ページ

それでもローカル：セキュリティは“値段がつかない”

課題

開発手法

実例

ローカルLLM

まとめ

機密会議で問われるのは「いくらか」ではなく「そもそも送ってよいか」

🚫 インサイダー情報・顧客の個人情報・規制対象データは、外部送信そのものが許されないことが多い

📊 IT責任者の53%が「データプライバシー」をAI活用の最大の障壁と回答（Cloudera 2025・約1,500名）

🕵️ 従業員の77%が会社情報をAIに貼り付け、その82%が個人アカウント経由＝“見えない漏洩”

➖ クラウドに一度送れば取り消せない（モデルに溶けたデータは消せない／EU当局2025）

✅ ローカルはデータが1バイトも外に出ない → GDPR・社内規程を自動的に満たす

コストはクラウド、安心はローカル。機密を扱う現場では“安心”が前提条件 — ローカルLLMの真価は「ローカルであること」そのもの

手法がもたらす経営価値

課題

開発手法

事例

ローカルLLM

まとめ

 **再現性**：判断が全部残る → 監査できる・属人化しない・引き継げる

 **床と天井**：誰がやっても下限品質を保証し、できる人ほど伸びる

 **横展開**：SynchroniNote で実証した手法を、他案件にそのまま適用できる

 **堀**：複数LLM運用力 + DD = 競合が素のAIでバイブコーディングしている限り、追いつけない

課題

開発手法

実例

ローカルLLM

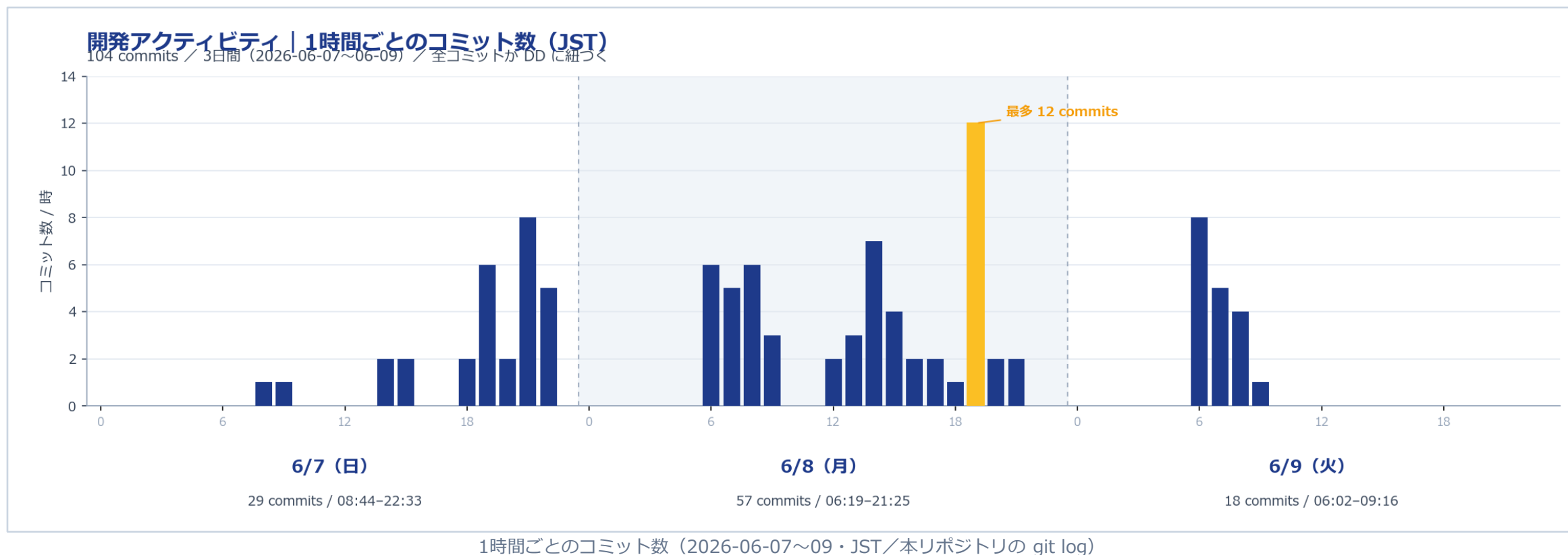
まとめ

- ナナイロは「AIで速く作る会社」ではなく、**「手法でAIを規律づけて作る会社」**
- その実例が、専門外から破綻なく完成した **SynchroniNote**（完全オフラインAI議事録）
- 手法（DD）と複数LLM運用力は、**他案件へ横展開できる組織の武器**

速さではなく、再現性。これがナナイロのものづくり。

付録：開発アクティビティ（コミット推移・JST）

- 日曜の朝に着手 → 月曜は早朝6時から夜21時台まで → 火曜の早朝に追い込み。104コミットすべてがDDに紐づく
「速い」のではなく、再現性ある手法に乗った“高密度な集中”の記録 — 一人の手で



付録：DD一覧（DD索引から自動生成）

全 51 件 — これらすべての「判断」が設計ドキュメントとして残る（監査でき・引き継げる）

DD-016 S-05リアルタイム画面の改善 …
DD-016-1 会話エリアの独立スクロールとヘッ…
DD-016-2 adhoc仮会議のライフサイクル…
DD-016-3 右パネル編集UIと状態結線と抽出…
DD-012-6 配布パッケージング Python…
DD-015 サイドメニュー最終形 DEV-W…
DD-014 P4-3ホットパスRust移植 …
DD-014-1 cpal收音とringbufとV…
DD-014-2 whisper-rs統合とAVX…
DD-014-3 Ollamaストリーム連携とTo…
DD-014-4 token-emit配線とDB書…
DD-013 P4-2協調編集CRDT AI追…
DD-013-1 Yjs導入とVue3バインディン…
DD-013-2 AI追記と人間入力の競合マージと…
DD-013-3 左右コピーUIとネットワーク同期…
DD-012 製品化 中核機能の実装と実用化
DD-012-1 マイク收音をライブでUIに接続

DD-012-10 事前資料ExcelPDF添付のテ…
DD-012-11 話者の永続化とS-03タイムライ…
DD-012-12 予定からライブ会議への実データ反…
DD-012-13 カレンダー予定のコピペ取込 テキ…
DD-012-14 入力デバイスの実列挙と切替 OS…
DD-012-2 会議終了から清書プレビュー保存の…
DD-012-3 SQLite永続化でカレンダーと…
DD-012-3-1 カレンダー確認用シードデータの整…
DD-012-4 LLM追い上げ整形のライブ表示
DD-012-5 話者ラベルの表示 DD004-1…
DD-012-7 S-08設定の永続化と実処理への…
DD-012-8 S-04プリフライトの実データ化…
DD-012-9 S-01カレンダー操作強化 削除…
DD-011 Phase4 Tauri2ペイン…
DD-010 準リアルタイム化 マイク收音と逐…
DD-010-1 長い発話の冒頭遅延と二段STT改…
DD-010-2 チャンク区切り品質改善 単語途中…

DD-009 画面設計SSOT モックを製品U…
DD-008 バッチMVP 音声から議事録Ma…
DD-008-1 評価期UI 音声から議事録の見え…
DD-007 データベース基本設計
DD-007-1 エンティティ抽出と永続化境界
DD-007-2 データ辞書統一
DD-007-3 論理設計ER図
DD-007-4 物理設計DDLとDB初期化
DD-007-5 DB設計の正式版昇格
DD-006 モデル切替シーケンス実測
DD-006-1 モデル切替実測の設計反映
DD-005 whispercpp実RTFクロ…
DD-004 話者分離PoC 日本語オフライン…
DD-004-1 ONNX話者埋め込みによる高精度…
DD-003 STT実機ベンチと同時実行スパイ…
DD-002 LLM生成速度ベンチマーク
DD-001 環境初期化 uvプロジェクト雛形…

付録：動作環境・技術スタック

- 実機：Windows 11 / Ryzen 7 PRO 8840HS (AVX-512) / GPUなし / RAM 59.7GB (実質約9.6GB)
- スタック：Tauri (Vue3 + Quasar) / Rust + Python評価ハーネス + Ollama
- 進め方：評価はPythonで速く回し、速度が要るホットパスだけRustへ移植
- 設計の正：企画書 → 開発ロードマップ → 基本設計書 (SSOT) → DD索引、と辿れる

付録：出典（ローカルLLM動向・コスト・安全性）

- Gemma 4 (Google, 2026-04-02) — blog.google / 26B MoE・活性約3.8B
- Qwen3 (Alibaba, 2025-04, Apache 2.0) — qwenlm.github.io
- オープン vs フロンティアの収束 — llm-stats.com / till-freitag.com (2026)
- Gemini API 料金 (2026) — ai.google.dev/gemini-api/docs/pricing
- オンプレLLMのプライバシー優位・Cloudera 2025調査 — unifiedaihub.com / digitalapplied.com

アクセス：2026-06。数値・料金は変動するため、提示時に最新を確認すること。